

intel_mgmt_master.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/host_if/intel_mgmt/intel_mgmt_master.vr

```
/*
 *
 * Author:      Fabian Delguste
 * File:        $RCSfile: intel_mgmt_master.vr,v $
 * Revision:    $Revision: 1.11 $
 * Date:        $Date: 2004/01/19 15:37:11 $
 *
 ****
 *
 * Basic Transaction Verification Module aimed at
 * creating read() and write() transactions based
 * on Intel management Interface
 *
 ****
 */

typedef class intel_mgmt_master;
class intel_mgmt_master_callbacks extends rvm_xactor_callbacks {
    // Called before a transaction is executed
    virtual task pre_trans_t(intel_mgmt_master master,
                            intel_mgmt_trans  tr,
                            var bit          drop)

    {}

    // Called after a transaction has been executed
    virtual task post_trans_t(intel_mgmt_master master,
                             intel_mgmt_trans  tr)

    {}
}

class intel_mgmt_master extends rvm_xactor {
    // Input rvm_channel: this is where transactions
    // to be sent stand
    intel_mgmt_trans_channel in_trans;

    // Event ID notifying when a transaction has been completed
    static integer TRANS_DONE;

    // Interface to Intel Bus HDL signals
    local intel_mgmt_port iport;
```

```

// Intel Mgmt configs
local intel_mgmt_cfg icfg;

// Constructor
task new(string          instance,
          integer        stream_id,
          intel_mgmt_port iport,
          intel_mgmt_cfg  icfg    = null,
          intel_mgmt_trans_channel in_trans = null);

// From rvm_xactor
virtual task reset_xactor(integer rst_type = 0);
protected virtual task main_t();

// Method to read the content of a specific address
// Generates corresponding pin-wiggling on the Intel Bus
local task read_t(var intel_mgmt_trans tr);

// Method to write data to a specific address
// Generates corresponding pin-wiggling on the Intel Bus
local task write_t(intel_mgmt_trans tr);

local function bit[7:0] sync_rdy_get_data_t();
}

task intel_mgmt_master::new(string          instance,
                             integer        stream_id,
                             intel_mgmt_port iport,
                             intel_mgmt_cfg  icfg    = null,
                             intel_mgmt_trans_channel in_trans = null) {
  super.new("Intel Mgmt Master", instance, stream_id);
  this.iport = iport;
  if (icfg==null) {
    icfg = new;
  }
  this.icfg = icfg;

  if (in_trans==null) {
    in_trans = new("Intel Mgmt Master Input Channel", instance);
  }
  this.in_trans = in_trans;

  this.TRANS_DONE = this.notify.configure(*, this.notify.ON_OFF_TRIGGER);

  this.iport.$Sel = 1 async;

```

```

    this.iport.$Rd    = 1 async;
    this.iport.$Wr    = 1 async;
    this.iport.$Data = 8'hZZ async;
}

task intel_mgmt_master::main_t() {
    intel_mgmt_trans tr;
    fork
        super.main_t();
    join none
    while(1) {
        bit drop = 0;

        // Get a transaction from the mailbox
        // and cast to a intel_mgmt_trans
        cast_assign(tr, this.next_transaction_t(this.in_trans));
        rvm_00_callback(intel_mgmt_master_callbacks, pre_trans_t(this, tr, drop));
        if (drop) {
            if (this.log.start_msg(this.log.DEBUG_TYP, this.log.DEBUG_SEV)) {
                if (this.log.text("Dropping transaction...")) {
                    void = this.log.text();
                    tr.display("  ");
                }
                void = this.log.text();
            }
            continue;
        }
        if (this.log.start_msg(this.log.DEBUG_TYP, this.log.TRACE_SEV)) {
            if (this.log.text("Starting transaction...")) {
                void = this.log.text();
                tr.display("  ");
            }
            void = this.log.text();
        }
        tr.notify.indicate(tr.STARTED);
        // Decide what to do now with the incoming
        // transaction
        case (tr.kind) {
            // Read cycle
            intel_mgmt_trans::READ: {
                read_t(tr);
                rvm_verbose(this.log, psprintf("Read Data=%2X @%2X",

```

```

        tr.data, tr.addr));
    }
    // Write cycle
    intel_mgmt_trans::WRITE: {
        rvm_verbose(this.log, psprintf("Writing Data=%2X @%2X",
                                       tr.data, tr.addr));

        write_t(tr);
    }
}

rvm_00_callback(intel_mgmt_master_callbacks, post_trans_t(this, tr));
this.notify.indicate(this.TRANS_DONE, tr);
tr.notify.indicate(tr.ENDED);
}
}

// Implementation of the read_t() method
//   - drives the address bus,
//   - select the bus,
//   - assert Rd signal,
//   - wait for an Rdy acknowledge
//   - read the data and return it.
// Note that this represent an asynchronous interface.
// Due to the asynchronism, there is no sensitive port
// used to trigger this method.
// It could be greatly simplified with synchronous I/F
task intel_mgmt_master::read_t(var intel_mgmt_trans tr) {
    // Drive Address bus
    this.iport.$Addr = tr.addr async;
    this.iport.$Wr = 1 async;
    // Wait for t1-t2, then assert Sel port
    delay(icfg.t1-icfg.t2);
    this.iport.$Sel = 0 async;
    // Wait for t2, then assert Rd port
    delay(icfg.t2);
    this.iport.$Rd = 0 async;
    // Get the data
    tr.data = sync_rdy_get_data_t();
    // Release Rd signal
    this.iport.$Rd = 1 async;
    this.iport.$Sel = 1 async;
    // Release the address bus
    delay(icfg.t5);

```

```

    this.iport.$Addr = 0 async;
    // Wait a little more prior to ending up the transaction
    delay(icfg.t5);
}
// Perform a write cycle
task intel_mgmt_master::write_t(intel_mgmt_trans tr) {
    // declare a dummy data variables which
    // contain the Rdy status
    bit Rdy;
    // Drive Address bus
    this.iport.$Addr = tr.addr async;
    this.iport.$Rd = 1 async;
    // Wait for t1-t2, then deassert Sel port
    delay(icfg.t1-icfg.t2);
    this.iport.$Sel = 0 async;
    // Wait for t2, then deassert Wr port
    delay(icfg.t2);
    this.iport.$Wr = 0 async;
    // Wait for t3-t10 then drive the data bus
    delay(icfg.t3-icfg.t10);
    this.iport.$Data = tr.data async;
    delay(icfg.t10);
    // If Rdy is still not asserted,
    // wait for it to be asserted low
    Rdy = this.iport.$Rdy async;
    if (Rdy != 0) {
        @(negedge this.iport.$Rdy async);
    }
    // Terminate the cycle
    this.iport.$Wr = 1 async;
    // Release Sel
    this.iport.$Sel = 1 async;
    // Release Addr
    delay(icfg.t5);
    this.iport.$Addr = 0 async;
    this.iport.$Data = 8'hZZ async;
    // Wait a little more prior to ending up the transaction
    // This allows the post_hook method to use the last Addr
    // rather the new one in case of consecutive transactions
    delay(icfg.t5);
}

```

```

}
// Wait until Rdy goes low (acknowledge) then
// wait for t4 ns and return the read Data
function bit[7:0] intel_mgmt_master::sync_rdy_get_data_t() {
    // declare a dummy data variables which
    // contain the Rdy status
    bit Rdy;
    delay(icfg.t6);
    // If Rdy is still not asserted,
    // wait for it to be asserted low
    Rdy = this.iport.$Rdy async;
    if (Rdy != 0) {
        @(negedge this.iport.$Rdy async);
    }
    // Sample the data after t4
    delay(icfg.t4+1);
    sync_rdy_get_data_t = this.iport.$Data async;
}

task intel_mgmt_master::reset_xactor(integer rst_type = 0) {
    super.reset_xactor(rst_type);
    this.in_trans.flush();

    this.iport.$Sel  = 1 async;
    this.iport.$Rd   = 1 async;
    this.iport.$Wr   = 1 async;
    this.iport.$Data = 8'hZZ async;
}

```